

gRPC Framework

- **What is gRPC and why does it work accross languages and platforms?**

It is an Open Source Framework for Remote Procedure Calls (functions can be controlled on a remote computer as if it was a local function call). It works across different languages, because it is standardized, like HTTP and Protocol Buffers.

- **Describe the RPC life cycle starting with the RPC client?**

The RPC client calls a method on a generated stub that looks like a local function. The stub serializes the request (usually with Protocol Buffers) and sends it over the network to the server. The server deserializes the request, executes the corresponding method, and prepares a response. The response is sent back, deserialized by the client, and returned as a local result.

- **Describe the workflow of Protocol Buffers?**

A developer defines data structures and services in a .proto file. This file is compiled using the protoc compiler, which generates source code for multiple languages. The generated classes handle serialization and deserialization of the data. Applications then use these classes to efficiently exchange structured information.

- **What are the benefits of using protocol buffers?**

Protocol Buffers are compact and fast due to their binary format. They provide strong typing and maintain forward and backward compatibility. They also allow automatic code generation for many languages, reducing boilerplate and errors. As a result, they improve performance and developer productivity.

- **When is the use of protocol not recommended?**

They are not recommended when data needs to be human-readable, since the binary format is not easily interpretable. For simple web APIs consumed primarily by browsers, JSON is often more convenient. They are also less

useful for very small or one-off data exchanges where their efficiency advantages are minimal.

- **List 3 different data types that can be used with protocol buffers?**
Examples include string, int32, and bool.